

《虫子的秘密生活》： 超越软件存储库中的错误和遗漏

豪尔赫·阿兰达
计算机科学系
多伦多大学
加拿大安大略省多伦多国王学
院路 10 号，邮编 M5S 3G4
jaranda@cs.toronto.edu

吉娜·维诺利亚
微软研究院
微软之路
美国华盛顿州雷德蒙德，98052
gina.venolia@microsoft.com

抽象的

每个 bug 背后都有一段故事。发现并解决 bug 的人员需要协调，从文档、工具或其他人那里获取信息，并解决责任、所有权和组织结构等问题。本文报告了一项关于 bug 修复协调活动的实地研究，该研究结合了案例研究和软件专业人员问卷调查。结果表明，即使是简单的 bug 的历史记录也强烈依赖于社会、组织和技术知识，而这些知识无法仅通过电子存储库的自动化提取，而且这种自动化提供的协调记录不完整，而且往往存在错误。本文利用丰富的 bug 历史记录和问卷调查结果来识别常见的 bug 修复协调模式，并为工具设计人员和软件开发协调研究人员提供参考。

通过问责制、所有权和组织结构等问题。流程中的每一步都存在意识要求、严重的效率低下以及提高生产力和质量的机会，但只有当我们超越抽象的数字，深入到每个 Bug 中丰富详细的协调历史中时，这些才会变得显而易见。

作为研究人员，我们经常依赖软件项目信息库作为提取错误和其他工作项历史记录的主要或唯一证据来源。它们通常以工单或记录的形式存储在错误数据库中。它们提供了便捷的工作划分。我们使用项目管理系统的功能，例如审计跟踪和数据字段，来跟踪每个工作项的所有权和上下文。有时，我们会利用团队成员之间的电子通信记录以及从人力资源数据库中提取的组织结构数据来丰富工单系统中的历史记录。

1. 简介

现代大规模软件开发需要每天管理大量的错误。修复错误是软件开发人员最常见且最耗时的工作之一 [15]。当错误数量达到数千时，团队成员和研究人员都难以记住它们的细节。抽象变得必不可少：项目健康状况以错误数量衡量；生产力则以修复的错误数量衡量。

在如此抽象的概念中，我们很容易忘记每个 bug 背后都有一个故事。发现并解决 bug 的人员需要协调，从文档、工具或其他人那里获取信息，并导航

然而，到目前为止，这些电子存储库作为错误或工作项历史记录的可靠和充分记录的用途尚未得到适当的验证，并且我们也没有对错误历史记录背后的协调动力学的描述。

本文报告了一项关于错误修复协调活动的实地研究，该研究结合了案例研究和软件专业人员问卷调查。这项研究超越了软件活动的电子存储库，直接与错误的关键参与者进行访谈，以发现通常用于修复错误的团队合作模式。本文探讨了电子存储库作为软件项目协调研究基础的可靠性，并为协调和意识工具的设计提供了一些启示。

2.相关工作

软件专业人员协作研究面临的问题是，有太多可能的事件和变量需要观察，而观察和分析资源却极其有限。选择性的需求贯穿于我们的数据收集和分析策略。为了解决这个问题，并且作为广义的概括，软件开发协作的实证研究通常遵循三种方法。

首先，如前所述，有些报告主要基于团队活动的电子追踪。这些电子追踪被用来替代对软件开发工作的观察。例如，Herbsleb 和 Mockus [10] 使用源代码变更管理系统数据（以及一项调查）来建模分布式软件开发。Cataldo 等人[4] 使用自动提取的档案数据来计算协调需求。Valetto 等人[19] 挖掘软件存储库以建立组织社会技术一致性的衡量标准。

其次，关于一位或几位软件专业人员的活动，存在着详尽而丰富的记录。这些记录通常是人种学观察和实地考察的结果，例如 Chong 和 Hurlbutt 关于编程结对协作的研究[5]，以及 Ko 等人[13]关于共置团队的信息需求。

第三，关于许多软件项目的报告更为广泛、更为抽象。这些报告往往也更加详细细致，但它们侧重于团队或项目层面上哪些地方做得对，哪些地方做得不对。例如，Brooks 对 OS/360 的经典事后分析 [3]，以及 Curtis 对认知、团队和组织动态的识别。等人[6]。最近，de Souza 和 Redmiles 采用了这种方法来表征软件专业人员的协调能力 [7]。

所有这些方法都是必要的，它们为我们提供了关于软件项目协调的非常有用的信息。但是，据我们所知，以工作单元为中心的协调研究方法在我们的领域中尚未被探索过。

其他学科中已有以工作单元为中心的协调研究的先例。常见的例子是根本原因分析 (RCA) [20] 的研究，该研究探索一系列流程故障，以发现并解决其根本原因。我们的方法论与 RCA 在许多方面有所不同；最重要的是，它并非着眼于定位流程中的主要故障点（甚至不需要发生故障），而是着眼于描述和表征流程本身。

软件开发中的协调研究通常将其描述为一种非正式、复杂且依赖于具体情境的现象。非正式和无记录的协调活动的重要性已被多次提及（尤其是 Kraut 等人

[14]，佩里等人[17]，以及 Herbsleb 和 Grinter [9]）。Lethbridge 对文档进行了研究，它是正式流程提案的核心。等人报告称，这些信息经常过时，而且开发人员很少参考[16]。但这并不意味着开发人员不需要不断获取项目信息：根据 Singer 的说法等人[18] 搜索是开发人员的主要工作之一。正如赫兹姆指出的 [11]，工程师们感兴趣的是找到 *值得信赖* 这些信息通常会引导他们去寻找同事或其他熟悉的来源，而不是文件。

最后，之前关于 Bug 库协调的研究已经从不同角度进行了探讨。它们是 Anvik 的 Bug 所有权自动分配研究的主要数据来源。等人

[1]。德索萨 等人[8] 研究了在美国国家航空航天局 (NASA) 的一个项目中管理相互依赖关系的方法，并将“问题报告”描述为边界对象，不同的软件专业人员群体将其用于不同的目的。

3. 研究设计和执行

我们的研究目标是提供一份内容丰富、情境化、以工作项为中心的关于错误修复任务协调的描述。我们主要有两个研究问题：

首先，软件团队是如何协调修复 bug 的？bug 的生命周期是怎样的？这项工作中最常见的协调模式是什么？随着时间的推移，以及在参与修复的团队的社会技术网络图谱中，bug 的解决效果如何？

其次，电子交互痕迹是否提供了足够好的协调图像，或者非持久性知识是否需要了解每个错误修复的故事？

我们进行了一项分两部分的实地研究。第一部分是关于 Bug 历史的多案例探索性案例研究 [21]。第二部分旨在通过对软件专业人员（开发人员、测试人员和项目经理）的调查来验证我们的案例研究结果。在这两个案例中，我们的数据均来自微软产品部门的软件开发。以下章节将分别描述我们研究的两个部分。

3.1 多案例研究

我们的案例研究的分析单位是*已修复错误的历史记录*。我们将其定义为概念上相关的活动的集合，这些活动在项目生命周期的某个时刻被汇总为错误数据库中的至少一个条目。错误数据库中的某些记录严格意义上来说并不是错误；我们仍然将它们视为错误，因为我们的团队就是这样做的。某些错误有重复的记录；我们将所有重复记录视为同一概念实体的一部分。某些错误表现出的症状最初被视为不同的错误并单独记录；我们尽可能将它们视为同一缺陷的一部分。

Bug 的生命期并不一定始于 Bug 数据库中创建条目之时，也不一定在标记为“已关闭”之时结束。有些 Bug 的生命期会双向延伸；这种延伸的生命期正是我们分析单位的一部分。

我们从微软的三个主要产品部门中挑选了案例。我们的选择标准是：

- 该错误被归档到错误数据库中，有关其性质的一些基本信息已发布在其数据字段中。
- 在我们开始观察时，该错误被标记为“已关闭”。
- 这个漏洞是新的（在我们开始观察后的两周内它就被关闭了）。

我们的案例是随机选择的。其他数据（例如错误的解决机制）不属于我们的选择标准，我们也没有对其进行控制。

为了更好地了解用户报告的错误的路径，我们在一种情况下偏离了我们的选择标准：我们联系了客户支持升级工程师并请求指向来自微软升级渠道的错误的指针。

我们所有的案例都遵循相同的方法。首先，我们查询产品部门的 Bug 数据库，找到符合我们标准的案例。我们从其电子记录中获取了尽可能多的信息，包括审计线索中的事件、Bug 记录的所有数据字段、Bug 所有者的数据以及参与该 Bug 相关行动的所有人的数据，以及源代码存储库的链接。

从那时起，我们通过联系最后接触或被错误记录引用的人员进行回溯。如果它们与历史记录无关（这种情况很常见，因为错误记录被批量编辑），我们会继续追溯，找到与该错误相关的人员。

一旦发现，我们就会采访他们，了解漏洞的历史以及他们从中获得的参与者和文物

信息或他们与之协调关闭的信息。当我们被指向某个工件（例如规范文档）时，我们会对其进行分析并追溯到其创建者。当我们被指向其他人时，我们会尽可能地联系他们，并与他们重复此过程。当我们被指向某个持久通信媒介（例如电子邮件）时，我们会获取一份副本，对其进行分析，并追溯到其发起者。

在某些情况下，这个过程会陷入僵局，但我们知道还有更多信息有待发现（例如，我们还没有找到最初发现 bug 的那个点）。如果是这种情况，我们会跳转到 bug 记录中的下一个相关参与者，并从该点继续重建 bug 历史记录。我们始终确保能够找到故事的开头。

在其他情况下，我们的调查会将我们引向事件链中距离我们已知的事件线索太远的人物和文物，以至于它们与我们漏洞的历史几乎没有关联。在这种情况下，我们会做出主观判断，停止探索这些分支。

我们的方法论在理论上受到了哈金斯分布式认知框架 [12] 对人、物件和信息流的关注的启发。然而，正如我们所预料的 [2]，我们发现了如此多的信息流实例，以至于在哈金斯研究的计算和表征层面上执行我们的案例研究并不可行。我们试图在丰富性和情境细节与可复制性和泛化能力之间取得平衡。当我们完全重建了一系列事件，或者部分重建了事件链，但由于缺乏部分参与者的参与或时间限制而无法继续进行，我们会停止收集数据。

我们的数据收集是半结构化的。针对每个案例，我们收集了以下信息：

- 历史上主要和次要人物及其贡献的列表。
- 相关工件和工具的列表。
- 按时间顺序列出错误历史记录中的信息流和协调事件。
- 根据每个案件的具体情况需要提供证据。
- 根据错误数据库中的记录重建的错误历史记录。
- 通过我们获得的全部电子痕迹重建的漏洞历史通过理解所有可用证据（包括我们对参与者的采访）重建的漏洞历史。

表 1 – 病例摘要

案件	案件类型	如何成立	解决	直接的代理商	间接代理商	其他听众	寿命 (天)	天带活动	活动
C1	文档	临时测试	固定的	6	4	179	320	12	19
C2	代码 (安全)	临时测试	固定的	21	6	3	408	49	138
C3	构建测试失败	自动化	固定的	四十二	8	291	59	21	141
C4	代码 (功能)	临时测试	固定的	6	1	0	7	5	16
C5	代码 (安装)	用户 (测试版)	按设计	2	3	0	2	2	12
C6	代码 (功能)	自动化	固定的	2	4	11	二十九	6	20
C7	构建测试失败	自动化	固定的	6	7	197	14	6	三十四
C8	代码 (功能)	狗粮	无法修复	2	7	0	2	2	5
C9	代码 (功能)	自动化	不可复制	5	2	1	2	2	12
C10	代码 (功能)	升级	固定的	23	18	十三	三十五	20	220

我们总共研究了十个漏洞（包括升级案例）。我们采访了26人。表1简要概述了我们的案例。*直接代理*是在解决 Bug 过程中执行任务的人；*间接代理*由直接代理处理，但没有干预。

案例研究中发现的代理与调查中报告的代理数量存在显著差异。调查中直接涉及代理的四分位数分别为 3（25%）、4（50%）、6（75%）和 15（100%）。相比之下，我们十个案例中有三个案例的直接涉及代理超过 15 名。我们认为，这种差异表明，我们的调查揭示的漏洞足迹比受访者认为的要大得多。

3.2 调查

我们通过一项针对微软软件专业人员的 54 个问题的调查问卷验证了案例研究的结果。我们将调查问卷发送给了 1,500 名随机选择的微软员工，这些员工平均分为开发人员、测试人员和项目经理。我们为参与者提供了赢取 500 美元礼品卡的机会。我们收到了 110 份回复（回复率为 7.3%）；所有回复均为可选，但所有问题都至少收到了 100 份回复。我们没有控制受访者的产品部门或人口统计标准。

调查询问了每位受访者关于最后的一个最近关闭的 bug，他们在解决过程中发挥了主要作用。要求他们查看 bug 数据库中的相应记录，并调出并重读所有与该 bug 相关的电子邮件，以便牢记该 bug 的历史记录。

调查主要分为三个部分。第一部分，受访者向我们提供了关于该 bug 的一般数据。第二部分，我们询问了我们将第 5 部分介绍的协调模式。第三部分，我们探究了 bug 数据库中的记录在多大程度上完整地描述了他们遇到的 bug。

图1-3展示了一些通用数据问题的结果。虽然用统计方法将案例研究与调查结果进行比较并不合适，但两者结果基本一致，只有一点例外：直接

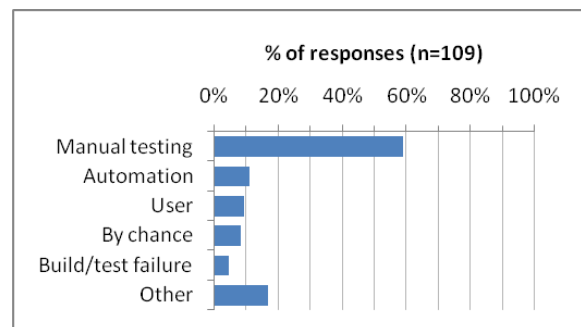


图 1-这个错误是如何发现的？

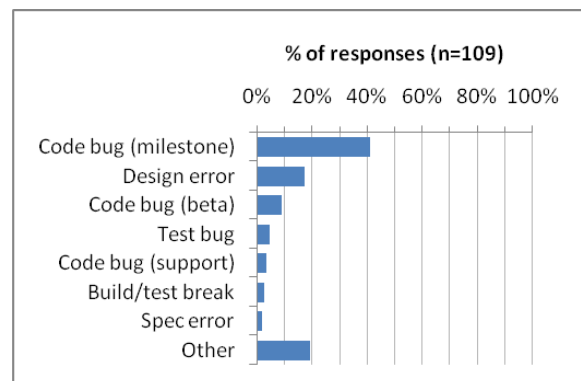


图 2 - 错误类型

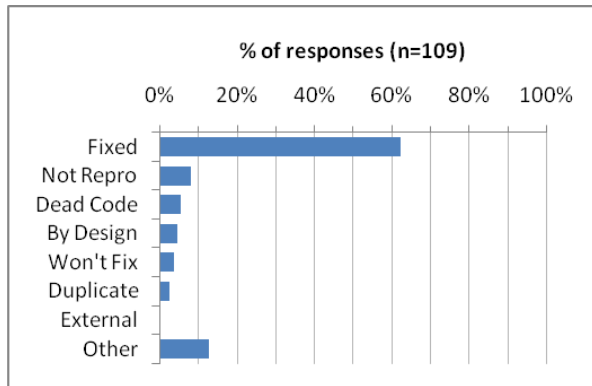


图 3-这个错误是如何解决的？

4. 错误和遗漏

我们从案例中得到的最深刻教训是电子追踪的不可靠性，尤其是错误记录，在我们十个案例中的七个案例中，这些记录是错误或误导性的，并且不完整和不足每一个案例。事实上，即使考虑到我们能找到的所有电子痕迹（存储库、电子邮件对话、会议请求、规范、文档修订和组织结构记录），在除一个案例外，其他所有案例历史记录省略了有关该错误的重要细节。

在讨论这些案例中的错误和遗漏之前，我们需要指出，我们认为这个问题并非微软特有的疏忽或缺乏纪律造成的。我们审查的存储库和文档似乎与同类公司一样详尽，甚至更胜一筹。我们将在“局限性”部分更详细地讨论这一点。

4.1 数据收集和分析的层次

我们发现，调查漏洞历史有多个级别，大致与每个级别需要投入的时间相对应。每个级别都包含了之前获取的所有信息，以及更深入分析的额外发现：

4.1.1 自动分析错误记录数据。在第一级，可以使用自动化来获取涉及错误历史的代理列表，以及诸如错误的生命周期、解决方案、状态变化、如何发现、所有者是谁、哪些代码变更集对应于该错误等信息，以及其事件的时间顺序列表。

4.1.2 电子对话和其他存储库的自动分析。电子对话的痕迹可以用来构建电子互动的社交网络，并假设其结构和强度与参与者的真实交流事件相对应。这些数据可以通过参与者、关键词和时间戳进行筛选，以定位可能与相关漏洞相关的电子互动。

4.1.3 人类的意义建构。自动化仍然远远达不到人类推断数据中每个联系和推理证据的能力。

首先，上述电子存储库中通常包含大量信息，但这些信息之间并没有正式的关联——发现这些信息需要对证据进行语义和非结构化的分析。例如，人员A在错误记录中可能指出，修复“无法解决X小组的性能问题，这些问题将提交到单独的错误记录中”。一个足够积极的人可能会得出结论，A和X小组的代表之间可能存在讨论，于是他从另一个数据库中提取X小组的人员列表，将其与电子邮件记录进行匹配，以确定相关的对话和涉及的代理，并通过反复试验查询（因为没有提供错误ID）在数据库中找到后续错误。

其次，这对自动化提出了更大的挑战，如果我们想要理解和改进协调机制，就需要将社会、政治和其他隐性信息纳入到我们的bug历史记录中，这些信息也是软件开发的核心内容。这些信息通常很微妙，并不总是显而易见，必须在收集到的证据中字里行间才能读懂。

4.1.4 参与者对历史的直接叙述。采访 Bug 历史记录的参与者似乎是获取未记录、不连贯或错误信息的最佳途径。只要采访在 Bug 历史记录被遗忘之前进行，就能极大地丰富之前阶段的数据。采访使我们能够验证先前的结论，并发现记录中未存储但对于理解 Bug 历史记录至关重要的事件。

4.2 实践中的级别

我们在列出的四个级别中的最后一个级别进行了案例研究。然而，在整个执行过程中，我们保留了与之前每个证据分析级别相对应的并行错误历史记录，以努力

确定在每个级别上获得或纠正了多少真实的错误历史记录。

不同级别之间的差异无论在数量上还是质量上都十分明显。下表有助于对比这些差异。表 2 显示了我们在错误历史记录中记录的每个级别的事件数量。同样，表 3 显示了在每个级别发现的代理（直接和间接）总数。

表 2 – 事件

案件	1级	2级	3级	4级
C1	8	16	17	19
C2	11	11	138	138
C3	19	119	119	141
C4	11	14	15	16
C5	8	11	11	12
C6	12	18	19	20
C7	6	33	三十四	三十四
C8	4	4	5	5
C9	7	11	12	12
C10	17	78	149	220

表 3 – 代理商

案件	1级	2级	3级	4级
C1	7	9	9	10
C2	5	5	二十七	二十七
C3	12	三十八	三十八	50
C4	5	5	7	7
C5	4	5	2	5
C6	7	7	5	6
C7	7	14	12	十三
C8	6	6	15	9
C9	6	7	7	7
C10	8	二十五	41	41

虽然这些数字支持了我们的观点，但它们无法展现不同层级之间 bug 历史记录的差异程度。这不仅仅是因为更高层级会产生更长的故事；更确切地说，它们会以与协调性研究密切相关的方式发生质的变化。以下段落描述了我们观察到的最重要的几种差异。

4.2.1 错误的数据库字段。错误记录中的基本数据库字段有时不正确。C9 是一个测试错误，但它被标记为代码错误。C2 和 C4 的一些重复项状态错误（它们的重复项已被标记为“已关闭”，但它们仍然被标记为“已解决”）。C8 被解决为“无法修复”，而它本应“按设计”解决。

我们的调查询问了参与者关于他们 bug 的“解决”字段以及他们的回复

支持我们的发现。10%的受访者表示他们的错误的“解决方案”字段不准确。

4.2.2 错误记录中缺少数据。错误记录中缺少的重要数据包括 C7 和 C10 中的源代码存储库链接、错误 C2 的重复和相关记录链接、C9 中解决原始错误过程中发现的错误的链接、任何规范文档的链接（特别是对于案例 C5，按设计解决）、重现步骤（C3）、修复错误所采取的纠正措施的声明（C2、C4、C7）以及错误根本原因的声明（C7、C9）。

缺少与源代码变更集的链接是最令人头疼的疏漏之一。对于我们调查中 70% 的受访者来说，修复最后一个 bug 需要将代码提交到代码库。但其中 23% 的案例中，bug 记录与源代码之间没有任何关联。

调查结果也印证了我们其他的发现。18% 的复现步骤被标记为不完整、不准确或缺失。而 bug 的根本原因和已采取的纠正措施的相应百分比分别为 26% 和 35%。

4.2.3 人员。获取bug历史中主要和次要参与者的列表经常会导致错误和遗漏。针对bug采取行动的人员通常不会在记录或电子邮件通讯中提及（C2、C3、C7、C9、C10）。所谓的bug所有者有时并未参与或参与解决bug（C6、C7、C8、C9）。基于电子痕迹，很容易误判参与者的贡献程度：高频率和高强度的互动并不意味着贡献程度高。至少有两次，员工数据库中受访者的地理位置信息存在错误。

在调查中，被标记为“bug 所有者”的人只有 34% 的时间在推动 bug 的解决。在 11% 的时间里，他们与 bug 无关。

此外，根据我们的反馈，10% 的情况下，通过查看错误记录很难找到主要参与修复错误的人员，10% 的情况下，他们甚至没有出现在记录中。40% 的情况下，编辑错误字段和历史记录的人员列表中只包含部分主要参与者，4% 的情况下，一个也没有。次要参与者的相应比例分别为 39% 和 38%。全部在 7% 的情况下，错误历史和字段中的人员完全不相关。

4.2.4 事件。期望与错误相关的所有事件都能在其记录中或通过其

电子痕迹。当然，大多数面对面的事件不会在任何存储库中留下任何痕迹。但在某些情况下，*钥匙Bug* 故事中的事件没有留下任何电子痕迹；发现这些痕迹的唯一方法是通过采访参与者。同时，我们所有案例的 Bug 记录中记录的一些事件都是噪音或垃圾（例如，批量编辑和错误以及之后的更正）。事件的时间顺序也存在问题：有时 Bug 甚至在创建记录之前就已解决（C7），或者在解决很久之后才关闭，因为需要关闭 Bug 的人已经忘记了它们（C2、C4）。

在调查中，3% 的漏洞在首次提交前至少一个月就已经被讨论过，另外 6% 的漏洞在提交前至少一周就已经被讨论过。

4.2.5 团体和政治。随着我们进一步从原始数据转向更广泛的协调模式，我们发现团队和部门层面的大多数重要信息只能通过更高层次的分析才能找到。在C4团队中，我们发现一小部分人的文化和实践与其所在部门不同，他们正处于被收购后的同化过程中。该团队在里程碑和发布方面的状态对发现新错误时做出的决策类型和速度有着重大影响（例如，在C5团队中，该团队正在进行“错误大扫除”，每天都要举行面对面的分类会议，大多数错误只被讨论了一分钟或更短的时间）。有时，就像在C3和C7团队中一样，错误的归属处于灰色地带，团队之间或部门之间会为确定归属和责任而产生争执。这些问题通常会对错误历史产生重大影响，但如果不采访参与者并密切关注电子记录中的细节，我们就无法了解这些问题。

4.2.6 基本原理。在没有人类理解和参与者协作的情况下，最难回答的问题可能是“为什么”这类问题：在C4中，为什么开发人员选择另一位开发人员作为必需的代码审查员，却选择第三位开发人员作为可选的审查员？在C10中，为什么在经过几次逐分钟更新和疯狂的电子邮件发送后，错误记录中却数周都没有任何活动？为什么C2、C4和C8中的“状态”或“解决”字段不正确？为什么在C5中，分类小组得出结论认为该错误无法修复？为什么测试人员在C9中提交了一个错误，即使她怀疑该故障很可能是误报？我们发现这些问题的答案，

在采访中发现，通常会解开错误历史事件的完整解释。

4.2.7 杂项。其他只有在更高层次的分析中才能发现的事实难以归类，但往往仍然是bug历史的核心。在C6案例中，一个bug由不同小组的测试人员和开发人员独立发现；开发人员在不知道该bug已被记录的情况下进行了修复。在C4案例中，一位开发人员在发现bug后不久就提交了数百行新代码来修复它；他并没有以惊人的速度编写所有代码，而是从他之前公司（现已被微软收购）的代码中复制代码，然后“拼凑”到相关的接口上。在两个案例（C3和C7）中，早期正确的诊断被迅速忽略，人们匆忙地发送邮件来处理紧急的bug。总的来说，我们案例池中的bug所包含的故事远比通过自动收集和分析其电子痕迹所能呈现的要丰富和复杂得多。

5. 协调模式

最终，我们的 bug 历史记录丰富多样，且与具体情况息息相关。它们并没有遵循统一的路径或生命周期。这就带来了一个问题：我们最初的研究目标恰恰是描述 bug 的生命周期及其修复过程。

我们没有尝试为所有 Bug 历史记录制定一个流程，而是选择描述我们观察到的协调模式菜单。我们选择了那些似乎最常出现的模式，以及那些很少出现但对 Bug 历史记录影响很大的模式。表 4 列出了这些模式。

有些模式会带来负面影响。例如，我们发现了一些“滚雪球式讨论”和“公开快速发送电子邮件”的情况，这些情况显然效率低下，但对我们的参与者来说，这似乎是家常便饭。我们观察到的唯一一次“峰会”对应一个 Bug（C3），负责的发布经理向我们描述该 Bug 时说它“非常重要”，并且“威胁到我们的发布日期”。为了确保完整性，我们添加了一个模式（视频会议），尽管我们没有观察到任何此类情况。另一个“被遗忘”的模式是由一位调查受访者指出的；它并没有被列入我们最初的列表中。

我们询问了受访者，这些模式是否在他们上一个 bug 中出现过，如果出现过，这些模式是否对 bug 的解决至关重要。图 4 展示了他们的回答。最后一列表模式的感知有用性与其出现频率的关系。

表 4 - 协调模式

传播媒体	
广播电子邮件	向多个邮件列表发送手动或自动通知，以告知其成员某个事件。
散弹邮件	向多个邮件列表和个人发送电子邮件，希望其中一个收件人能够解答当前问题
滚雪球线程	将人员添加到不断增加的电子邮件收件人列表中。
探索专业知识	向一个或几个人发送电子邮件，而不是通过“散弹枪”式的方法，希望他们要么拥有专业知识来协助解决问题，要么可以将电子邮件转发给可以解决问题的人。
探究所有权	向一个或几个人发送电子邮件，而不是通过“散弹枪”方法，请求他们接受该错误的所有权或可以将其重定向到愿意接受的人。
不频繁的直接电子邮件	少数人之间不频繁地私下发送的电子邮件。
快速电子邮件	在解决问题的过程中，少数人私下里爆发了大量电子邮件活动。
在公共场合快速发送电子邮件	与上述情况类似，但有数十或数百人被抄送为电子邮件线程的收件人，其中大多数人与该问题无关。
即时通讯讨论	使用即时通讯平台传递信息、排除故障或联系他人。
电话	电话交谈用于传递信息、排除故障或联系他人。
错误数据库	
关闭-重新打开	由于错误诊断或解决，或者对解决方案存在分歧或团队推迟解决该问题的能力存在分歧，因此重新打开该错误。
已提交后续错误	在修复此错误的过程中发现并提交了其他错误，或者将此错误的一部分提交到不同的记录中作为后续处理。
被遗忘	长期未被注意和无人关注的错误记录。
编写代码	
代码审查	该错误的修复已由至少一位同行审核并批准。
一石二鸟	修复此错误还修复了之前发现和提交的其他错误。
当我们在那里时	修复此错误还修复了之前未发现的其他错误。
会议	
顺便来你的办公室	与附近办公室的同事非正式地面对面获取信息或交流有关该问题的一些想法。
状态会议中的播出时间	该问题在定期小组状态会议上进行了讨论。
乱堆	这个问题需要团队召开一次专门会议来讨论。
首脑	这个问题需要不同部门的人员召开会议专门讨论。
远程会议参与者	任何至少有一名成员远程参加的会议（可能是小组会议或峰会）。
视频会议	任何使用视频与至少一名与会者进行交流的会议（可能是小组会议或峰会）。
其他图案	
忽略修复/诊断	早期提出的正确诊断或解决方案暂时被大多数人忽略。
回避所有权	收回错误或代码的所有权。
分类	讨论并决定这是否是一个值得解决的问题。
参考规范	至少有一个对规范、设计文档、场景或愿景声明的具体和特定的引用，以提供解决或解决问题的指导。
意外的贡献	来自讨论该问题的群体之外的人们的新信息或替代方案。
深度合作	两个或两个以上的人密切合作（面对面或电子方式）并持续一段时间来解决问题。

	Essential (E)	Occurred (O)	(E / O)
Probing for ownership	16%	30%	52%
Summit	1%	2%	50%
Probing for expertise	17%	34%	49%
Code review	30%	66%	46%
Triaging	32%	69%	46%
Rapid-fire email	13%	33%	38%
Infrequent, direct email	18%	49%	37%
Shotgun emails	7%	19%	37%
Drop by your office	23%	64%	36%
IM discussion	11%	32%	35%
Phone	7%	20%	33%
Meetings w/remote part.	4%	12%	33%
Huddle	2%	6%	33%
Air time in status meeting	8%	27%	29%
Close-reopen	4%	13%	29%
Broadcasting emails	10%	35%	27%
Referring to the spec	6%	24%	24%
While we're there	3%	14%	21%
Rapid-fire (in public)	1%	5%	20%
Follow-up bugs filed	3%	18%	16%
Deep collaboration	3%	22%	13%
Snowballing threads	2%	16%	13%
Two birds with one stone	1%	17%	6%
Ownership avoidance	1%	28%	3%
Ignored fix/diagnosis	0%	9%	0%
Video conferences	0%	3%	0%
Unexpected contribution	0%	9%	0%

图 4 - 模式的实用性和频率

我们并不声称我们的模式列表是全面的。但是，使用它们来描述错误历史，或许可以提供关于其协调事件的足够相关信息，同时忽略无关细节。此外，正如我们将在下一节中讨论的那样，其中一些模式可以通过软件工具来支持（或者在负面模式的情况下，可以预防）。

6. 含义

6.1 对于工具设计师

虽然 Bug 数据库中常用的状态集（活跃、已分配、已解决等）可能有助于管理软件开发，但我们的数据表明，它们并不能很好地反映 Bug 的真实生命周期。我们认为，为了理解协作机制，并为开发人员设计工具，更有用的做法是，不要将 Bug 修复活动视为

属于错误生命或工作流程的一个阶段，但努力实现一个或多个目标。

我们根据案例研究中人员的活动制定了一份目标清单。表 5 对这些目标进行了描述。并非所有案例都会出现这些目标，而且这些目标的发生顺序也并非严格一致。

这些目标提供了一个框架来分析协调和项目管理工具及实践的有效性。例如，*所有权转让*通常会带来问题，尤其是在看似有缺陷的代码没有明确的所有者的情况下。在案例研究中，这种情况在测试脚本中比在功能代码中更常见，而且开发人员经常在未完成工作的情况下就离职。确保每个工件都有活跃所有者的工具和实践可以减少这个问题。

*搜索*是另一个问题所在。在我们的案例中，它常常导致“滚雪球式”的邮件和“散弹式邮件”。虽然有时能找到所需的人员或信息，但如果考虑到数百名邮件收件人需要耗费大量人力来解析大量与他们无关的邮件，那么这种做法的效率可能极其低下。

为了协调目的，*意识*是最需要改进的领域。然而，在产品互联的大型公司中，如何提供适当的认知水平并非易事。最需要认知的似乎并非团队层面，而是围绕工作项目构成社交网络的主要和次要代理。一些能够（部分）检测工作网络并允许其成员了解同事活动的工具应该有助于解决这个问题。

6.2 对于研究人员

我们的案例研究和调查结果都指向同一个方向：电子存储库中保存的数据往往不完整或不正确。我们基于对错误数据库和电子邮件通信的探索得出这一结论；源代码存储库比这些数据源更难对付，因为它们忽视了测试人员、项目经理、可用性专家和用户网络，而这些人也是协调现象的主要推动者。

我们发现的一些具体差异，对于大规模自动化协调分析来说，或许是可以接受的。其他一些差异，例如 4.2.3 版本中的大部分人员问题，以及从错误记录到源代码变更集的缺失链接，则要严重得多。

表 5 – 利益相关者在 bug 生命周期内的目标

发现	检测现实与预期之间的差异。将意外行为记录为 Bug 是至关重要的第一步。
诊断	了解漏洞的性质、原因和影响，以及后续将采取的措施。我们相信，这种情况在每种情况下都会发生，尽管通常是不言而喻的。
所有权转让	确定谁将负责解决错误，包括小组和个人层面。
搜索	寻找合适的知识、资源和技能。这似乎经常与所有权分配活动交织在一起，因此专家是 Bug 的所有者——但即使是所有者也可能需要寻求其他更具体的专业知识和技能。
更正	我们通常认为的“修复 bug”是指修改相关的代码行、文档、脚本或其他内容，使现实与预期再次匹配。
关闭	确定组织是否愿意接受与该错误相关的当前状态。
意识	与相关参与者沟通状态。它涵盖了几乎每个 Bug 的整个历史记录。

可以采取多种措施来减少电子存储库带来的问题。至关重要的第一步是将它们连接起来。同样重要的是，为自动化工具提供一些基本的智能，以检测和忽略噪声活动和批量编辑。最后，研究参与组织的标准操作可以帮助研究人员确定大多数协调活动是否留下可挖掘的电子痕迹。

但即使解决了上述所有问题，仅通过自动化得出有关协调动力学的结论仍有可能无法察觉大规模软件开发中的个人、社会和政治因素，从而错过构成软件产品的每个交互的本质。

7. 限制

在分析过程中，我们探讨了几个在文献中尚未有统一定义的概念。具体来说，有人可能会认为我们的协调模式和目标主观且界限模糊——例如，我们从未明确区分“快速发送”和“不频繁发送”的电子邮件。虽然这种批评有理有据，但考虑到我们收集的数据，我们的构想只是初步迭代。更多数据和进一步的迭代应该能够完善这些构想，并添加其他有助于更清晰地表达底层概念的构想。

我们的数据全部来自微软，如果不进行重复实验，我们的研究结果对其他公司的适用性尚不明确。众所周知，微软拥有数万名员工、数百万日活跃用户以及众多互联互通的产品。这些都是塑造协作的力量。

动态。然而，微软对软件存储库和通信媒体的使用似乎与其他同类公司类似。我们研究中最明显的发现，即漏洞历史的可挖掘版本与真实版本之间的差异，不应是微软独有的，因为它并非取决于企业文化，而是取决于能够经济高效地以电子方式获取的信息的数量和质量。

对于某些软件开发环境，尤其是开源软件，所有或大部分信息都以电子方式持续传输，这可能不是什么大问题。重复这项研究将有助于解答这个问题。

8. 结论

这项实地研究发现，即使是简单的漏洞历史，也高度依赖于社会、组织和技术知识，而这些知识无法仅通过软件存储库的自动分析来提取。自动提取的历史记录会提供事实错误以及不完整且错误的协调记录。本文探讨了一些缓解此问题的策略。

该研究还利用丰富的错误历史记录，发现了许多错误修复的协调模式，并通过对软件专业人员的调查进行了验证。通过分析参与这些模式的人员的根本目的，我们得出了八个错误修复目标，这些目标可以作为设计更好的工具和实践的框架。

尽管我们最初考虑协调功能开发和错误修复，但试点功能案例（本文未报告）被证明属于完全不同的领域，并且

我们选择专注于修复错误。我们计划在未来的工作中探索这两种开发活动之间的差异和互动。

致谢

我们感谢微软研究院的 HIP 团队，以及 Steve Easterbrook 和 Greg Wilson 在整个研究过程中进行的富有成效的讨论。我们还要感谢参与我们研究的软件专业人员所付出的时间。本文第一作者曾于 2008 年暑期在微软研究院实习。

参考

[1] J. Anvik, L. Hiew 和 GC Murphy. 谁应该修复这个 bug? 在 *ICSE'06: 第28届国际软件工程会议论文集*, 第 361-370 页, 上海, 中国, 2006 年。

[2] J. Aranda 和 SM Easterbrook. 软件工程研究中的分布式认知: 它能发挥作用吗? 在 *首届支持大规模软件开发社会方面的研讨会 (SSSLSSD)*, 加拿大班夫, 2006 年。

[3] F.P.布鲁克斯。《人月神话》.Addison Wesley, 1995 年。

[4] M. Cataldo, PA Wagstrom, JD Herbsleb, 和 KM Carley. 协调需求的识别: 对协作和意识工具设计的启示。在 *CSCW'06: 第20届计算机支持协同工作会议论文集*, 第 353-362 页, 加拿大班夫, 2006 年。

[5] J. Chong 和 T. Hurlbutt. 结对编程的社会动态。在 *ICSE'07: 第29届国际软件工程会议论文集*, 第 354-363 页, 明尼阿波利斯, 明尼苏达州, 美国, 2007 年。

[6] B. Curtis, H. Krasner, 和 N. Iscoe. 大型系统软件设计过程的实地研究。 *ACM 通讯*, 31(11): 1268-1287, 1988。

[7] CRB de Souza 和 DF Redmiles. 一项关于软件开发人员依赖关系和变更管理的实证研究。在 *ICSE'08: 第30届国际软件工程会议论文集*, 第 241-250 页, 德国莱比锡, 2008 年。

[8] CRB de Souza, DF Redmiles, G. Mark, J. Penix 和 M. Sierhuis. 协作软件开发中的相互依赖关系管理。在 *ISESE 2003: 2003 年国际经验软件工程研讨会论文集*, 第 294-303 页, 2003 年。

[9] JD Herbsleb 和 RE Grinter. 拆分组织与集成代码: 再论康威定律。在 *ICSE'99: 1999 年国际软件工程会议论文集*, 第 85-95 页, 美国加利福尼亚州洛杉矶, 1999 年。

[10] JD Herbsleb 和 A. Mockus. 全球分布式软件开发中速度和通信的实证研究。 *IEEE 软件工程学报《细胞与分子免疫学杂志》* 2003 年 第29卷第6期

[11] M. Hertzum. 信任在软件工程师评估和选择信息源中的重要性。 *信息与组织*, 12(1): 1-18, 2002。

[12] E.哈金斯。 *野外认知*。麻省理工学院出版社, 1995 年。

[13] AJ Ko, R. DeLine 和 G. Venolia. 共置软件开发团队中的信息需求。 *ICSE'07: 第29届国际软件工程会议论文集*, 第 344-353 页, 明尼阿波利斯, 明尼苏达州, 美国, 2007 年。

[14] RE Kraut 和 LA Streeter. 软件开发中的协调。 *ACM 通讯*, 38(3): 69-81, 1995。

[15] TD LaToza, G. Venolia 和 R. DeLine. 维护心智模型: 一项关于开发人员工作习惯的研究。在 *ICSE'06: 第28届国际软件工程会议论文集*, 第 492-501 页, 上海, 中国, 2006 年。

[16] TC Lethbridge, J. Singer 和 A. Forward. 软件工程师如何使用文档: 实践现状。 *IEEE 软件*, 20(6): 35-39, 2003。

[17] D. Perry, NA Staudenmayer 和 LG Votta. 人员、组织和流程改进。 *IEEE 软件*, 11(4): 36-45, 1994。

[18] J. Singer, T. Lethbridge, N. Vinson, 和 N. Anquetil. 对软件工程师工作实践的考察。在 *CASCON'97: 1997 年合作研究高级研究中心会议论文集*, 加拿大安大略省多伦多, 1997 年。

[19] G. Valetto, M. Helander, K. Ehrlich, S. Chulani, M. Wegman 和 C. Williams. 利用软件存储库调查开发项目中的社会技术一致性。在 *MSR'07: 第四届软件存储库挖掘国际研讨会*, 美国明尼苏达州明尼阿波利斯, 2007 年。

[20] PF Wilson, LD Dell, GF Anderson. *根本原因分析: 全面质量管理的工具*. ASQ 质量出版社, 1993 年。

[21] 尹红克. *案例研究: 设计和方法 (第三版)*. Sage, 2003 年。